

Implementing AI on a Computer

“What if a computer could understand what a human wants, instead of requiring the human to learn how to operate the computer?”

The Core Idea

Computers have traditionally required the human to learn their interface — which folder to open, which menu holds a setting. This approach reverses that: the human states a goal in natural language, and the AI determines how to operate the computer to achieve it.

Human states the goal → AI interprets the goal → Computer executes the work.

The agent is not a voice assistant layered on top of a PC. It is built on a Raspberry Pi running Linux, with the reasoning handled by a large language model — Claude or OpenAI's models are both viable.

Two Modes of Operation

The computer runs in two modes: a standard Linux mode, functioning as an ordinary desktop with keyboard, mouse, monitor, and any Linux application, with no AI involvement — and an AI mode, triggered by a wake word, in which a stated goal is interpreted, planned, and executed automatically.

Example invocations:

- “Find my sales report and open it.”
- “Create a folder called Customer Reports.”
- “Prepare the sales report and send it to my boss.”

System Architecture

The reasoning model operates on a continuous loop. Each cycle factors in the result of the previous action before deciding the next one, rather than following a fixed, predetermined sequence:

Hear → Understand → Plan → Act → Observe → (repeat)

Execution happens on the Raspberry Pi itself, through two complementary methods. Most actions route through a small set of deterministic, pre-written functions — finding a file, creating a folder, renaming or moving a file, opening an application, reading a file, or running an approved script. The model only selects which function to call and what parameters to pass; it never touches the operating system directly.

For applications with no dedicated function, a fallback method takes a screenshot, decides on an action, clicks or types accordingly, and verifies the result with a new screenshot before continuing:

*Screenshot → Decide the action → Click or type → Verify with a new screenshot
→ Repeat until done.*

This fallback is slower and less deterministic than a direct function call, so it is used only when nothing more specific is available.

Hardware

The system runs on a Raspberry Pi 5 as the main processing unit, with a USB microphone for voice capture and wake-word detection, and a USB speaker for spoken responses — USB rather than a 3.5mm jack, since the Pi 5 does not have one. A microSD card stores the OS, applications, and files; Wi-Fi or Ethernet provides the network link to cloud AI APIs; a display, keyboard, and mouse are only needed for standard-mode use or initial setup; and an enclosure houses everything as a dedicated unit. API credentials are stored separately from network configuration and used to authenticate with the AI service.

End-to-End Task Example

Instruction: “Prepare the sales report and send it to my boss.”

- Locate the required sales data.
- Open the spreadsheet application.
- Prepare the report.
- Save the completed report.
- Prepare the message and attach the report.
- **Request confirmation** before the sensitive external action (sending it).
- Send the message after confirmation, and verify delivery.

A single high-level instruction resolves into a completed, multi-step task.

Safety & Control

Autonomy does not imply absence of control.

- **Confirmation prompts** before destructive or external actions (deleting files, sending messages).
- **Restricted permissions** — the agent never has more access than the task requires.
- **Secure credential storage** for API keys.
- **Audit logs** recording every action attempted and taken.
- **Fail-safe behavior** — the system stops and asks rather than guessing when uncertain.

- **Remote dashboard** (optional) — lets an authorized user observe activity and status from another device.

Computers have historically required humans to adapt to their operation. This architecture inverts that relationship — not through additional interface layers, but by building goal interpretation directly into how the machine operates.

Instead of humans learning how to operate computers, what if computers could learn how to understand humans?